01 prefix sum

Prefix Sum Pattern

1. What is it, and when is it used?

A **prefix sum** (a.k.a. cumulative sum) is a precomputed array `pre[]` where `pre[i] = arr[0] + arr[1] + ... + arr[i]`. Once built, the sum of any subarray `[l, r]` can be answered in **O(1)** as `pre[r] - pre[l-1]`.

It is used whenever a problem asks you to repeatedly answer **range sum / range aggregate queries** over a static (or rarely-changing) array, or when you need to detect subarrays whose sum satisfies some condition (equals K, divisible by K, etc.).

2. Why is it used?

Without it, computing the sum of a subarray takes O(n) per query, and if you have Q queries the total cost is O(n·Q). Prefix sum flips this to **O(n) preprocessing + O(1) per query**, which is the difference between TLE and AC on most range-query problems.

3. Where is it used?

- Range sum queries (static arrays)
- Subarray sum equals K / divisible by K
- Equilibrium index / pivot index problems
- 2D matrix range sum queries (2D prefix sum)
- Difference arrays for range **update** problems (the inverse trick)
- Counting subarrays with a given XOR, product constraints (via prefix XOR / prefix product+log)
- Balancing problems: max score from removing K elements from ends (prefix + suffix sums)
- Product of array except self (prefix product × suffix product)

4. How do you identify it's a Prefix Sum problem? (Pattern Triggers)

Say to yourself **"is this asking about a running total/aggregate over a contiguous range, computed many times?"** If yes → prefix sum.

Signal phrases (the 2-minute test): - "sum of subarray from index i to j" repeated many times - "number of subarrays with sum equal to K" - "subarray whose sum is divisible by K" - "equilibrium / pivot index" (left sum == right sum) - "range sum query" / "range update query" (mutable → Fenwick/segment tree, immutable → prefix sum) - "maximum/minimum sum by removing k elements from front and back" - "count subarrays with equal number of 0s and 1s" (convert 0→-1, then prefix sum = 0 repeats)

5. Can you identify it from constraints?

Constraint clue	Implication
Multiple queries (Q up to 1e5) on a static array	Precompute prefix sum → O(1)/query
Array is immutable	Plain prefix sum array is enough
Array has updates interleaved with queries	Need Fenwick Tree / BIT or Segment Tree instead
n, Q ≤ 1e5–1e6 and asks "count subarrays with property X"	HashMap + prefix sum, O(n)
2D grid with multiple sub-rectangle sum queries	2D prefix sum

6. Types / Variants

1. **1D Prefix Sum** — classic running sum array.
2. **2D Prefix Sum** — for rectangle sum queries in a matrix.
3. **Prefix XOR** — for subarray XOR queries.
4. **Prefix Product** — product of array except self, range product (careful with zeros).
5. **Difference Array** — inverse of prefix sum; used for O(1) range *updates*, then prefix-sum once at the end to materialize.
6. **Prefix Sum + HashMap** — to count subarrays with sum == K in O(n), using `map[prefixSum - K]`.
7. **Prefix + Suffix Sum combo** — for two-ended problems (take k elements from either end).

7. Rationale / Intuition

The key insight: `sum(l, r) = totalSum(0, r) - totalSum(0, l-1)`. Any "sum over a range" is just "the difference of two prefix sums." This converts an O(n) range computation into an O(1) lookup, at the one-time cost of O(n) preprocessing. The HashMap variant extends this: if `pre[j] - pre[i] == K`, then `pre[i] == pre[j] - K`, so instead of checking every pair (O(n²)), we just look up how many earlier prefix sums equal `pre[j] - K` (O(n)).

8. Time & Space Complexity

Operation	Time	Space
Build prefix array	O(n)	O(n)
Range sum query (after build)	O(1)	—
Subarray-sum-equals-K (HashMap)	O(n)	O(n)
2D prefix build	O(n·m)	O(n·m)
2D range query	O(1)	—

9. Comparison with Other Patterns

Pattern	Best for	Handles updates?	Query time
Prefix Sum	Static array, range sum queries	No	O(1)
Difference Array	Static queries, many range *updates*, one final read	Range updates yes, point queries only after final build	O(1) update, O(n) to materialize
Sliding Window	Contiguous subarray with monotonic constraint (all positive, no removal needed)	N/A	O(n) total
Fenwick Tree (BIT)	Range sum with point updates	Yes	O(log n)
Segment Tree	Range sum/min/max with range or point updates	Yes	O(log n)
Kadane's	Max subarray sum (no queries, single pass)	N/A	O(n)

Use prefix sum when the array doesn't change and you need many range queries or subarray-sum-count problems. Move to Fenwick/Segment tree the moment updates enter the picture.

10. Reusable Java Templates

10.1 Basic 1D Prefix Sum

```
class PrefixSum {
    private final long[] prefix;
```

```
        public PrefixSum(int[] arr) {
            prefix = new long[arr.length + 1];
            for (int i = 0; i < arr.length; i++) {
                prefix[i + 1] = prefix[i] + arr[i];
            }
        }

        // inclusive range [l, r], 0-indexed
        public long rangeSum(int l, int r) {
            return prefix[r + 1] - prefix[l];
        }
    }
```

10.2 Subarrays with Sum Equal to K (HashMap + Prefix Sum)

```
    import java.util.HashMap;
    import java.util.Map;

    class SubarraySumEqualsK {
        public int subarraySum(int[] nums, int k) {
            Map<Long, Integer> freq = new HashMap<>();
            freq.put(0L, 1); // empty prefix
            long sum = 0;
            int count = 0;

            for (int num : nums) {
                sum += num;
                count += freq.getOrDefault(sum - k, 0);
                freq.merge(sum, 1, Integer::sum);
            }
            return count;
        }
    }
```

10.3 2D Prefix Sum (Matrix Range Sum)

```
    class NumMatrix {
        private final long[][] prefix;

        public NumMatrix(int[][] matrix) {
            int rows = matrix.length, cols = matrix[0].length;
            prefix = new long[rows + 1][cols + 1];
            for (int r = 0; r < rows; r++) {
                for (int c = 0; c < cols; c++) {
                    prefix[r + 1][c + 1] = matrix[r][c]
                            + prefix[r][c + 1] + prefix[r + 1][c] - prefix[r][c];
                }
            }
        }

        public long sumRegion(int r1, int c1, int r2, int c2) {
            return prefix[r2 + 1][c2 + 1] - prefix[r1][c2 + 1]
                    - prefix[r2 + 1][c1] + prefix[r1][c1];
        }
    }
```

10.4 Difference Array (Range Update, then materialize)

```
    class DifferenceArray {
        private final long[] diff;

        public DifferenceArray(int n) {
```

```
            diff = new long[n + 1];
        }

        // add 'val' to every index in [l, r]
        public void rangeUpdate(int l, int r, long val) {
            diff[l] += val;
            diff[r + 1] -= val;
        }

        // call once after all updates to get the final array
        public long[] materialize() {
            long[] result = new long[diff.length - 1];
            long running = 0;
            for (int i = 0; i < result.length; i++) {
                running += diff[i];
                result[i] = running;
            }
            return result;
        }
    }
```

11. Quick Recognition Checklist (2-minute test)

- ☐ Does the problem mention “range” or “subarray” sums repeated over many queries?
- ☐ Is the array static (no interleaved updates)?
- ☐ Is it asking to count/find subarrays satisfying a sum condition?
- ☐ Would brute force be $O(n^2)$ or $O(n \cdot Q)$ and TLE on given constraints?

If 2+ boxes checked → strongly a Prefix Sum problem.